SDM120_Web – Stromzähler-Anzeige im Browser über ESP32-Access-Point

Ein **Eastron SDM120** (Modbus RTU über RS485) wird von einem **D1 Mini ESP32** ausgelesen.
Der ESP32 spannt ein eigenes WLAN auf und liefert eine Webseite mit allen Messwerten aus.
Ein Handy, Tablet oder Laptop verbindet sich mit diesem WLAN und öffnet die Seite im Browser.
Es braucht **kein Internet, keinen Router und keine App**.

```
SDM120 ──A/B (RS485)── RS485-TTL-Modul ──UART2── D1 Mini ESP32 ))) WLAN-AP ))) Browser
```

---

1. Dateien

Datei	Zweck
`SDM120_Web.ino`	Firmware für den ESP32 (Arduino IDE), Webseite ist eingebettet
`index.html`	Dieselbe Webseite als Einzeldatei. Lokal per Doppelklick geöffnet, läuft sie im **Demo-Modus** mit simulierten Werten (zum Testen der Optik ohne Hardware)
`README.md`	Diese Dokumentation

> **Hinweis:** Die Webseite steht zweimal im Projekt: als `index.html` und eingebettet in der `.ino`. Wenn du die Optik änderst, ändere `index.html`, teste sie lokal und kopiere den Inhalt dann zwischen `R"rawliteral(` und `)rawliteral"` in die `.ino`.

---

2. Hardware

Bauteile

- D1 Mini ESP32 (ESP32-WROOM-32)
- RS485-TTL-Modul: am besten **MAX3485** (3,3 V) oder ein Modul mit **automatischer Richtungsumschaltung**.
  Ein klassisches MAX485-Modul läuft mit 5 V. Dann braucht die RO-Leitung zum ESP32 einen Spannungsteiler (z. B. 1 kΩ / 2 kΩ).
- Eastron SDM120 **Modbus**-Version (SDM120M / SDM120-Modbus)
- Verdrillte Zweidrahtleitung für A/B (bei längeren Strecken ein 120-Ω-Abschluss am Leitungsende)

Verdrahtung

RS485-Modul	D1 Mini ESP32	Bemerkung
RO	GPIO16 (RX2)	Empfangen

RS485-Modul	D1 Mini ESP32	Bemerkung
DI	GPIO17 (TX2)	Senden
DE + RE (gebrückt)	GPIO4	Richtungsumschaltung. Bei Modulen mit Auto-Richtung: im Code `PIN_DE = -1`
VCC	3V3 bzw. 5V	je nach Modul, siehe oben
GND	GND	
A	SDM120 Klemme **A**	Beschriftung am Zähler beachten
B	SDM120 Klemme **B**	

> ⚠ Der SDM120 hängt an 230 V. Der Anschluss der Netzseite gehört in die Hand einer Elektrofachkraft.
> Die RS485-Klemmen sind vom Netz getrennt, trotzdem nur im spannungsfreien Zustand verdrahten.

3. Einstellungen im Code

Alles steht oben in `SDM120_Web.ino` im Block **Konfiguration**:

Konstante	Standard	Bedeutung
`AP_SSID`	`SDM120-Zaehler`	Name des WLANs
`AP_PASS`	`zaehler123`	WLAN-Passwort, mind. 8 Zeichen. `""` ergibt ein offenes WLAN
`DNS_UMLEITEN`	`true`	Jede eingegebene Adresse führt zur Zählerseite. Handys zeigen die Seite oft automatisch als „Anmelden im Netzwerk“ an
`SDM_ID`	`1`	Modbus-Adresse des Zählers (Werkseinstellung 001)
`SDM_BAUD`	`2400`	Baudrate (Werkseinstellung SDM120: 2400, 8N1)
`PIN_RX` / `PIN_TX`	`16` / `17`	UART2-Pins
`PIN_DE`	`4`	DE/RE-Pin, `-1` = automatische Richtung
`TIMEOUT_MS`	`250`	Maximale Wartezeit auf eine Antwort
`PAUSE_MS`	`60`	Pause zwischen zwei Modbus-Anfragen

Auf der Webseite (Script-Anfang in `index.html`):

Konstante	Standard	Bedeutung

Konstante	Standard	Bedeutung
`INTERVALL_MS`	`2000`	Wie oft die Seite neue Werte holt
`P_MAX_W`	`3680`	Vollausschlag des Leistungsbalkens (16 A × 230 V)
`HIST_PUNKTE`	`150`	Länge des Verlaufs (150 × 2 s = 5 min)

Baudrate und ID lassen sich am SDM120 selbst über das Setup-Menü prüfen und ändern (Taste lange drücken, Ablauf siehe Zähler-Handbuch).

---

4. Flashen (per USB, ohne OTA)

1. Arduino IDE → Boardverwalter → **esp32 by Espressif** installieren (Core 2.x oder 3.x).
2. Board: **WEMOS D1 MINI ESP32** (oder „ESP32 Dev Module“).
3. Ordner `SDM120_Web` öffnen. Die `.ino` muss in einem gleichnamigen Ordner liegen.
4. Port wählen und hochladen. Zusätzliche Bibliotheken braucht es nicht (`WiFi`, `WebServer`, `DNSServer` sind im Core enthalten).
5. Seriellen Monitor auf **115200 Baud** stellen. Alle 10 s erscheint eine Statuszeile:

```
U=231.4 V  I=1.87 A  P=412 W  E=1234.567 kWh  ok=5123  fehler=0
clients=1
```

---

5. Benutzung

1. Am Handy mit dem WLAN **SDM120-Zaehler** verbinden.
2. Wenn das Handy meldet „Kein Internet“: **verbunden bleiben** wählen.
3. Browser öffnen → `http://192.168.4.1`. Dank DNS-Umleitung funktioniert auch jede andere Adresse.

Die Seite zeigt:

- **Wirkleistung** groß, mit Balken. Blau = Bezug, grün = Einspeisung (der SDM120 meldet Einspeisung als negative Leistung).
- Spannung, Strom, Frequenz, Leistungsfaktor, Schein- und Blindleistung, Phasenwinkel
- Energie: Bezug, Einspeisung, Gesamt (kWh)
- Leistungsverlauf der letzten 5 Minuten. Er wird im Browser gesammelt und beginnt beim Neuladen der Seite von vorn.
- Statuspunkt: **grün** = Zähler online · **gelb** = ESP32 erreichbar, Zähler antwortet nicht · **rot** = ESP32 nicht erreichbar

---

6. Funktionsweise

Modbus-Abfrage

- Funktion **0x04 (Read Input Registers)**, je Messwert 2 Register = 1 Float (IEEE 754, Big Endian).
- Pro Durchlauf von `loop()` wird **nur ein Register** gelesen. So bleibt der Webserver reaktionsfähig, obwohl 2400 Baud langsam sind. Alle 11 Werte sind nach etwa 1,5–2 s einmal komplett aktualisiert.
- Antworten werden auf Länge, Adresse, Funktionscode und CRC16 geprüft. Exception-Antworten werden erkannt.
- Ein Wert, der länger als 10 s nicht erfolgreich gelesen wurde, wird als `null` gemeldet und auf der Seite als „—“ angezeigt.

Register-Tabelle

Adresse	JSON-Name	Größe	Einheit
0x0000	`u`	Spannung	V
0x0006	`i`	Strom	A
0x000C	`p`	Wirkleistung	W
0x0012	`s`	Scheinleistung	VA
0x0018	`q`	Blindleistung	var
0x001E	`pf`	Leistungsfaktor	–
0x0024	`phi`	Phasenwinkel	°
0x0046	`f`	Frequenz	Hz
0x0048	`eImp`	Wirkenergie Bezug	kWh
0x004A	`eExp`	Wirkenergie Einspeisung	kWh
0x0156	`eGes`	Gesamt-Wirkenergie	kWh

Weitere Register lassen sich einfach ergänzen: eine Zeile in `REG[]` hinzufügen, dazu ein Feld in `index.html`.

JSON-API

`GET http://192.168.4.1/api/werte`

```
{
  "id": 1,
  "werte": { "u": 231.4000, "i": 1.8700, "p": 412.0000, "s": 432.7000, "q":
132.1000,
             "pf": 0.9520, "phi": 17.8000, "f": 50.0100,
             "eImp": 1234.5670, "eExp": 12.3450, "eGes": 1246.9120 },
  "ok": true,
  "alter_ms": 1650,
  "stat": { "ok": 5123, "fehler": 0, "laufzeit_s": 932 }
}
```

Feld	Bedeutung
`ok`	`true`, wenn alle Werte aktuell sind
`alter_ms`	Alter des ältesten gelieferten Werts
`stat.ok` / `stat.fehler`	Zähler erfolgreicher bzw. fehlgeschlagener Modbus-Lesungen seit dem Start
`stat.laufzeit_s`	Laufzeit des ESP32 in Sekunden

Die API sendet `Access-Control-Allow-Origin: *`. Andere Programme im selben WLAN können sie also direkt abfragen,
z. B. eine Delphi-/TMS-Web-Core-Anwendung oder eine App-Inventor-App über die Web-Komponente.

---

7. Fehlersuche

Symptom	Ursache / Abhilfe
Statuspunkt gelb, alle Werte „—“, `fehler` steigt	A/B vertauscht → tauschen. Baudrate/ID am Zähler prüfen. RX/TX vertauscht. DE/RE nicht angeschlossen oder `PIN_DE` falsch
Nur einzelne Werte fehlen sporadisch	Leitung zu lang oder ungeschirmt → 120-Ω-Abschluss setzen, `TIMEOUT_MS` auf 400 erhöhen
Werte völlig unsinnig	Falsche Byte-Reihenfolge ist beim SDM120 nicht zu erwarten. Wahrscheinlich ist ein anderes Gerät mit derselben ID am Bus
WLAN nicht sichtbar	Passwort kürzer als 8 Zeichen → AP startet nicht. Seriellen Monitor prüfen
Handy trennt das WLAN	In den WLAN-Einstellungen „Verbindung ohne Internet halten“ aktivieren bzw. mobile Daten kurz abschalten
Seite lädt, Statuspunkt rot	Handy hat das WLAN gewechselt. Wieder mit `SDM120-Zaehler` verbinden

---

8. Erweiterungsideen

- Zusätzlich ins Heim-WLAN einbuchen (`WIFI_AP_STA`) für Fernübertragung, z. B. per MQTT
- Tages-/Monatsverbrauch aus `eImp` berechnen und im Flash (Preferences) sichern
- Mehrere Zähler am selben Bus (verschiedene IDs) auf einer Seite